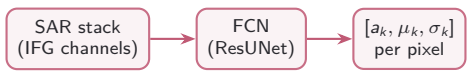
DLR-TomoSAR — Project Status

Improvements, Additions, Future, Results

July 2026



Amortise per-pixel Gaussian-mixture fitting of the elevation spectrum in one forward pass.

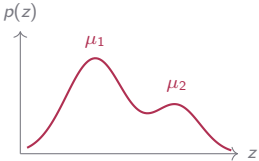
The task

The elevation power spectrum at each pixel is a K-Gaussian mixture:

$$p(z) = \sum_{k=1}^{K} a_k \exp\Big(-\tfrac{(z-\mu_k)^2}{2\sigma_k^2}\Big)$$

Classical processing fits this per pixel by nonlinear least squares. We train a network to emit all $3K$ parameters directly.

$$f_\theta : \mathbb{R}^{C_{\text{in}} \times P \times P} \longrightarrow \mathbb{R}^{3K \times P \times P}$$



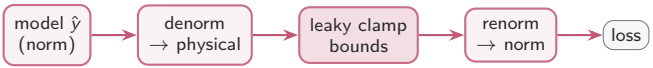
Two scatterers at μ_1, μ_2.

1 Improvements

leaky clamp · robust normalization · per-group LR & warmup

Improvement — Output clamp (leaky clamp)

The model predicts in *normalized* space, so the physical bound is enforced by a round-trip: denormalize, clamp, renormalize — the loss is then taken back in normalized space.



A saturated head must still receive a gradient toward the valid range:

$$\tilde{x} = \mathrm{clamp}(x) \;+\; \lambda\,\big(x - \mathrm{clamp}(x)\big).\mathrm{detach}()$$

Gradient never fully zero past the bound.

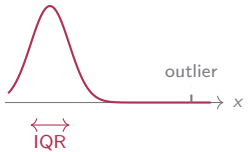
- Forward = hard clamp; a fraction $\lambda = 0.01$ of the residual leaks gradient (straight-through).
- Physical bounds: $a \in [0, a_{\max}]$, $\mu \in [x_{\min}, x_{\max}]$, $\sigma \in \left[\tfrac{1}{2}x_{\text{step}}, \tfrac{1}{2}x_{\text{range}}\right]$.
- Inference uses $\lambda = 0$ (plain hard clamp).

Improvement — Robust (IQR) normalization

Heavy-tailed channels switched from z-score to *robust* scaling:

$$\hat{x} = \frac{x - \mathrm{median}(x)}{\mathrm{IQR}(x)}, \qquad \mathrm{IQR} = Q_3 - Q_1$$

- Median / IQR ignore the tails, so rare bright scatterers no longer inflate the scale ($+12.. + 39\sigma$ under z-score).
- Applied to `mag`, `amp`, `sigma`, `dem` (with `log1p` first); `mu` stays z-score.
- Stats pooled over *active* pixels only ($a > 10^{-2}$).



Scale set by the bulk, not the tail.

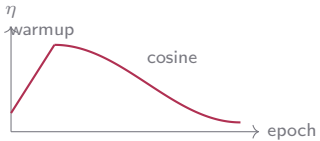
Improvement — Per-group LR + warmup

Each submodule gets its own base LR; a step-level warmup ramps in, and a cosine schedule anneals per epoch:

$$\eta_{\text{eff}} = \eta_0 \cdot \underbrace{F(t)}_{\text{cosine}} \cdot \underbrace{f_{\text{warm}}(s)}_{\text{ramp}}$$

$$f_{\text{warm}}(s) = \alpha_0 + (1 - \alpha_0)\frac{s}{S}$$

- Groups: encoder / decoder $= 3 \times 10^{-4}$, output head $= 10^{-3}$ (head learns faster).
- Warmup: $S = 200$ steps, start $\alpha_0 = 0.1$; cosine floor $\eta_{\min} = 10^{-6}$.
- LR auto-scales with batch size.



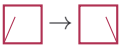
2 Additions

augmentation · Hungarian matching · model $\times$ loss benchmark

Addition — Augmentation & Hungarian matching

Augmentation

- Joint H/V flips ($p = 0.5$) on input+target (keeps spatial correspondence).
- Optional 90° rotation (off by default; skipped when geometry present).
- Input-only Gaussian noise, in normalized units.



Hungarian matching

$$\pi^\star = \arg\min_{\pi \in S_K} \sum_k w_k \left\| \hat{p}_k - p_{\pi(k)} \right\|_1$$

- Default training matcher: permutation-invariant slot alignment.
- Brute-force over $K!$ perms ($K \leq 6$), vectorised over pixels (*not* `scipy`); inactive GT slots excluded.
- Removes the "Gaussian k in slot k" constraint.

Addition — Slot rebalancing (ships with Hungarian)

Active normalization
Reduce the parameter loss as a weighted mean over *active* slots only:

$$\mathcal{L} = \frac{\sum_k m_k \ell_k}{\sum_k m_k}, \quad m_k = \mathbb{1}[a_k^{\mathrm{GT}} > \tau]$$

Empty slots (GT mean ≈ 1.28 of 6) stop diluting the gradient.

Presence balance
Inverse-frequency reweighting of the amplitude term, per batch:

$$w_{\mathrm{act}} = \frac{1}{\rho}, \quad w_{\mathrm{inact}} = \frac{1}{1-\rho}, \quad \rho = \frac{\#\mathrm{active}}{\#\mathrm{slots}}$$

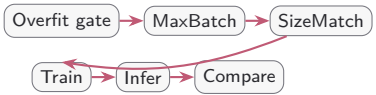
Under-represented active Gaussians get up-weighted automatically.

These two are the full model's only slot-presence terms — *no* presence head, BCE, or focal.

When to use it: Hungarian + balance is a *high-K* strategy — it pays off when many Gaussians compete for slots. At $K = 2$ the simpler `sort_gt_by_mu` matching wins, so it stays the default there.

Addition — Model × loss benchmark

Capacity-matched sweep: 10 architectures at ≈30M params, identical recipe, each × several loss components.



Curve-level R^2 (top of field)

Model	R^2
resunet	**0.593**
unet++	0.555
attention_unet	0.542
unet	0.530
transunet / unetr	≈0.42

CNN U-Nets lead; transformer hybrids trail at matched capacity.

3 Future

loss curriculum · physical (SAR) loss terms

Future — Loss curriculum

Warm up with a simple loss, then swap to the full composite at a fixed epoch, optionally resetting the schedule, warmup, and Adam moments.

- Both phases use Hungarian matching + active-norm + presence-balance.
- Checkpoint baseline auto-resets across the (non-comparable) loss scale.

Future — Physical (SAR) loss terms

Match the predicted reflectivity to the observed interferometric covariance via the steering operator, rather than only in parameter space:

$$\hat{R}_{ij} = \int p(z)\, e^{j(k_{z,i} - k_{z,j})z}\, dz, \qquad k_z = \frac{4\pi\, b_\perp}{\lambda\, r_0}$$

$$\mathcal{L}_{\text{phys}} = \left\| \hat{R}(\hat{p}) - \hat{R}(p^{\text{GT}}) \right\|_F^2$$

- Ranked: (1) covariance matching, (2) coherence re-synthesis; moments as a metric.
- k_z from perpendicular baselines; steering sign settled ($+k_z$, 0.67m median peak error).
- `total_power` rejected as a loss (Capon $\sim$2 dB scene bias).

4 Results

16 architectures $\times$ 6 loss functions $\cdot$ 96 runs $\cdot$ curve-level R^2

Result — Benchmark design

A joint **architecture** × **loss** sweep: **16 architectures** each trained under **6 loss functions** (**96 runs**), all capacity-matched to ≈30M parameters under one identical recipe and passed through the same staged gates — so each cell isolates the architecture × loss pairing.

16 architectures

single-head backbones only (the multi-head / per-Gaussian variants and `unet_skip` are excluded):

- unet, resunet, attention_unet, unet++, linknet
- swin_unet, transunet, unetr, deeplabv3+, segformer
- convnext_unet, dense_unet, hrnet
- multires_unet, fpn, u2net

6 loss functions

Loss	Kind	Space
`l1_curve`	L1	curve
`mse_curve`	MSE	curve
`param_l1`	L1	param
`param_mse`	MSE	param
`charbonnier_curve`	Charb.	curve
`cosine_curve`	cosine	curve

L1 & MSE in both spaces, plus Charbonnier and cosine.

Result — Curve-level R^2 grid

Model	$L1_c$	MSE_c	$L1_p$	MSE_p	Charb	Cos
unet	–	–	–	–	–	–
resunet	–	–	–	–	–	–
attention_unet	–	–	–	–	–	–
unet++	–	–	–	–	–	–
linknet	–	–	–	–	–	–
swin_unet	–	–	–	–	–	–
transunet	–	–	–	–	–	–
unetr	–	–	–	–	–	–
deeplabv3+	–	–	–	–	–	–
segformer	–	–	–	–	–	–
convnext_unet	–	–	–	–	–	–
dense_unet	–	–	–	–	–	–
hrnet	–	–	–	–	–	–
multires_unet	–	–	–	–	–	–
fpn	–	–	–	–	–	–
u2net	–	–	–	–	–	–

Curve-level R^2 on the held-out test split (higher is better); best cell to be highlighted. *Numbers to be filled from the benchmark run.*

Summary

Improvements	leaky clamp · IQR norm · per-group LR + warmup
Additions	augmentation · Hungarian + rebalancing · benchmark
Future	loss curriculum · physical SAR loss
Results	16 architectures × 6 losses · curve-level R^2 grid